Algorithmic Strategies for 1v1 Poker Bots: Heuristic and Opponent Modeling Approaches in Highly Constrained Environments

1. Introduction: The Imperative for Rapid-Deployment Architectures

The development of autonomous agents for Heads-Up No-Limit (HUNL) Texas Hold'em has historically been dominated by heavy computational paradigms designed to approximate a Nash Equilibrium. In academic and competitive environments, such as the Massachusetts Institute of Technology (MIT) Pokerbots competition (officially designated as Course 6.9630 during the January Independent Activities Period or IAP), participants are tasked with engineering a highly competitive poker bot within an aggressively compressed timeframe of roughly one month. For engineering teams entering the development cycle with a strict one-week deadline, traditional state-of-the-art algorithms are structurally infeasible.

Counterfactual Regret Minimization (CFR) and Deep Reinforcement Learning (DRL) currently represent the apex of unexploitable, Game Theory Optimal (GTO) poker AI. However, CFR requires traversing game trees containing upwards of 10^{160} decision points, necessitating complex state abstractions and weeks of continuous self-play on supercomputing clusters. For instance, landmark bots like Libratus required millions of CPU hours on supercomputers to train. Furthermore, standard CFR relies heavily on bucket-based decision-making—grouping similar hands into discrete "buckets"—which introduces abstraction translation errors, loss of granularity, and requires extensive iterative tuning. Similarly, DRL architectures suffer from high variance, delayed reward sparsity, and catastrophic forgetting without meticulously designed neural network topologies and lengthy training regimens.

When constrained by a one-week development sprint and stringent runtime limits—such as a maximum of 1GB of RAM, a 100MB compressed file size, and a strict 30-second time bank to execute 1,000 hands (leaving mere milliseconds per decision)—developers must pivot toward lightweight, exploitative, and computationally inexpensive architectures. This research report details an exhaustive framework for building a highly competitive 1v1 poker bot that explicitly eschews CFR, DRL, and bucket-based abstractions. The focus is directed entirely toward mathematically grounded rule-based heuristics, dynamic Effective Hand Strength (EHS) evaluations, Bayesian opponent modeling, and real-time Miximax/Monte Carlo sampling algorithms.

2. Exploitative Play Versus Game Theory Optimal (GTO)

To construct a competitive bot in a short timeframe, it is necessary to establish the theoretical foundation of the agent's strategy. Poker bots generally fall into two distinct categories: Game-Theoretic bots and Exploitative bots.

Game-Theoretic bots, powered by CFR, attempt to play a strategy that minimizes exploitability. They seek a balanced, mixed strategy that guarantees a non-negative expected value (EV) regardless of the opponent's strategy. While robust, GTO play possesses a critical flaw in

competitive settings against sub-optimal opponents: it assumes the opponent is also playing perfectly. Real-world opponents, including amateur human players and heuristically flawed bots, routinely deviate from GTO play. A pure GTO bot fails to capitalize on these specific mistakes, opting instead to maintain its own defensive balance.
Conversely, Exploitative bots are designed to identify and systematically attack the strategic leaks of their opponents. If an opposing bot is observed to fold too frequently to aggression, an exploitative bot will dynamically increase its bluffing frequency to capture dead money in the pot. If an opponent calls too loosely, the exploitative bot will eliminate its bluffing range and exponentially increase its value-betting sizing. Given the 1v1 nature of the MIT Pokerbots competition and the high likelihood of facing imperfect student-built agents, deploying an exploitative architecture driven by real-time opponent modeling will reliably yield a higher expected win rate than a hastily constructed, poorly abstracted CFR approximation.
3. Pre-Flop Strategic Heuristics: Deterministic Baselines
In No-Limit Texas Hold'em, the pre-flop phase is the most frequently encountered decision node. Because the game tree has not yet branched into the near-infinite complexities of community cards, pre-flop decisions can be almost entirely deterministic. By relying on pre-calculated mathematical heuristics rather than live simulations, the bot preserves its highly limited 30-second time bank for the computationally intensive post-flop scenarios.

3.1 The Chen Formula Evaluation

To avoid the necessity of maintaining massive lookup tables for pre-flop hand ranges, the Bill Chen Formula serves as a highly efficient, algorithmic method to quantify the intrinsic value of any starting hand in real-time. Instead of mapping 169 strategically distinct starting hands to a hard-coded database, the Chen Formula calculates a dynamic score based on high-card value, pair bonuses, suitedness, and connectedness.
The evaluation metrics are structured sequentially to yield a numerical score that dictates the agent's opening action:

Evaluation Metric	Scoring Mechanism
High Card Base Score	The highest card in the hand dictates the base score. Ace = 10, King = 8, Queen = 7, Jack = 6. Cards 10 through 2 are scored as exactly half their pip value (e.g., a 10 is worth 5 points, an 8 is worth 4 points).
Pair Bonus	If the two hole cards form a pair, the base score of the single card is multiplied by 2. A minimum score of 5 is guaranteed for any pair (e.g., pocket 2s score a 5, not a 2).
Suited Bonus	If the two hole cards are of the same suit, 2 points are added to the total score, accounting for flush potential.
Closeness (Gap) Penalty	Points are subtracted based on the gap between the cards to account for diminished straight potential. No gap (e.g., 8-9) = 0 penalty. 1-card gap (e.g., 7-9) = -1 point. 2-card gap = -2 points. 3-card gap = -4 points. 4-card gap or more = -5 points.

Evaluation Metric	Scoring Mechanism
Low Straight Bonus	If the gap is 0 or 1, and both cards are lower than a Queen (e.g., J-T, 8-6), 1 point is added. This compensates for the mathematical reality that lower connectors can form straights in both ascending and descending directions.

By implementing the Chen Formula, the bot can instantly categorize its starting hand into actionable tiers without utilizing RAM-heavy data structures. A programmatic implementation allows the bot to map scores directly to action probabilities. For example, a score below 6 typically triggers an immediate fold from early positions or against raises. Scores between 7 and 9 justify a standard call or a positional open-raise. Scores of 10 or above justify aggressive 3-betting or 4-betting. This continuous scoring mechanism entirely circumvents the need for discrete, rigid bucketing abstractions while maintaining mathematical rigor.

3.2 Sklansky-Chubukov Rankings for Short-Stacked Play

As 1v1 matches progress and the effective stack sizes diminish relative to the escalating blinds, post-flop play becomes a mathematical impossibility. The Stack-to-Pot Ratio (SPR) drops so low that any continuation bet commits the entirety of a player's stack. In these scenarios, the bot must seamlessly transition from traditional multi-street value betting into a purely push/fold (all-in or fold) pre-flop strategy.

The Sklansky-Chubukov (SC) rankings provide a rigorous mathematical framework for determining the exact maximum effective stack size at which moving all-in pre-flop is theoretically profitable. The defining characteristic of the SC rankings is their worst-case-scenario assumption: the rankings calculate the Chip-Expected Value (cEV) of an all-in move assuming that the opponent has perfect knowledge of the bot's hole cards and will only call if they are mathematically favored to win.

Because the SC rankings assume an omniscient opponent, any all-in move made below the SC threshold is unconditionally profitable. The opponent's actual lack of perfect information only increases the fold equity and the ultimate profitability of the move.

Starting Hand	SC Max Effective Stack (in Big Blinds)	Strategic Implication for the Agent
AA / KK	\infty	Unconditionally profitable to push regardless of stack depth.
AKs	~100 BB	Profitable to open-shove up to deep-stack thresholds.
K4o	11 BB	Counter-intuitively profitable to push at 11 BBs due to the mathematical weight of fold equity and blockers.
86s	4.5 BB	Push only if the effective stack is extremely shallow.

Incorporating the SC matrix into the pre-flop logic allows the bot to execute perfect, unexploitable short-stack strategies without requiring a CFR-generated push/fold chart. The logic gate is straightforward: when the bot detects that the effective stack size has dropped below 15 big blinds, it shifts from the Chen Formula evaluator to querying the SC maximum

stack boundary. If the current stack is lower than the SC value for the dealt hand, the bot immediately executes an all-in maneuver, completely eliminating the need for post-flop computation in shallow-stack scenarios.

4. Post-Flop Architecture: Effective Hand Strength (EHS) and Monte Carlo Rollouts

Once community cards are revealed across the flop, turn, and river, static heuristics like the Chen Formula become obsolete. The bot must accurately quantify the absolute strength of its hand relative to the board texture, while simultaneously accounting for the unknown distribution of the opponent's hand. To bypass the necessity of massive neural network value estimation functions, the bot architecture must rely on the Effective Hand Strength (EHS) algorithm bolstered by highly optimized Monte Carlo simulations.

4.1 The Mathematics of Effective Hand Strength (EHS)

The core assumption of the EHS algorithm, pioneered in the late 1990s by the Computer Poker Research Group (CPRG) during the development of the Loki and Poki bots, is that a poker hand's true value is a composite of two factors: its immediate strength at the current moment, and its potential to either improve or degrade as future community cards are dealt.
The EHS calculation relies on extracting three distinct mathematical metrics from the game state:

1. **Hand Strength (HS):** The probability that the bot's current hand is stronger than the opponent's hand, assuming no more cards will be dealt. It is calculated by enumerating all possible two-card combinations the opponent could hold and determining the ratio of wins and ties. For example, against a random hand on the flop, there are \binom{47}{2} = 1,081 possible opponent combinations. If the bot's hand wins against 444 combinations, ties against 9, and loses to 628, the HS is evaluated as 0.585.
2. **Positive Potential (PPOT):** The mathematical probability that a hand currently losing to the opponent's range will improve to become the winning hand on subsequent streets (analogous to implied odds). This is calculated by simulating the remaining undealt board cards and tracking the transition of losing states into winning states.
3. **Negative Potential (NPOT):** The mathematical probability that a hand currently winning will be overtaken by the opponent on subsequent streets (analogous to reverse implied odds). For instance, holding a made straight on a flop featuring two hearts carries a high NPOT, as a third heart on the turn or river would complete an opponent's flush draw.

These variables are synthesized into the final Effective Hand Strength equation. In its simplest form, it models the current strength plus the chance of improving if behind:
However, in advanced heuristic implementations, the formula is expanded to explicitly subtract the statistical risk of losing the lead:
.

4.2 Monte Carlo Approximations for Time-Constrained Environments

Calculating the exact EHS on the flop requires a full enumeration of all possible future board rollouts. Given that there are 47 unknown cards in the deck, a two-card lookahead from the flop to the river requires evaluating the 1,081 possible opponent hands against the \binom{45}{2} = 990 possible remaining board runouts, yielding over 1.07 million distinct evaluations per decision node.
Operating under a strict 30-second time bank for 1,000 hands renders exhaustive deterministic enumeration computationally fatal; the bot would exhaust its time allowance within the first few

hands. To circumvent this bottleneck, the architecture utilizes Monte Carlo simulations. Rather than traversing the entire probability space, the engine randomly samples a subset of possible opponent hands and future board cards.
Research implementations, such as the *Bluffasaurus* bot and the *hughpearse* MIT Pokerbots submission , demonstrate that simulating between 2,000 and 3,000 random match iterations yields an EHS convergence that is statistically indistinguishable from a full enumeration, while executing in under 0.1 seconds per decision. The win frequency generated by the Monte Carlo rollout is mapped directly to a value between 0.0 and 1.0.

4.3 Pot Odds and Rate of Return Triggers

To translate this raw EHS equity into a concrete action, the bot continuously calculates the Pot Odds. The Pot Odds formula is defined as C / (C + P), where C is the cost required to call a bet and P is the current total size of the pot.
If the Monte Carlo-derived EHS is mathematically greater than the Pot Odds (EHS > \frac{C}{C+P}), the bot has a theoretically profitable continuation. From here, the bot calculates an expected Rate of Return. High rates of return (where EHS heavily dominates Pot Odds) trigger an aggressive value bet or raise; moderate rates trigger a passive check or call; low rates trigger an immediate fold.

5. The Core of Exploitative Play: Bayesian Opponent Modeling

While EHS and Monte Carlo simulations provide a robust mathematical baseline, utilizing them against a completely random distribution of opponent hands is inherently sub-optimal. Real-world opponents, especially custom-built student bots, play specific styles and reveal information through their betting patterns. The most critical differentiator in a 1-week heuristic bot is its capacity to track these patterns and exploit them through real-time Opponent Modeling.

5.1 Tracking Action Frequencies and Probability Triples

To model an opponent without the heavy computational overhead of Long Short-Term Memory (LSTM) neural networks , the bot maintains a localized, statistical database of the adversary's tendencies. The most fundamental mechanism is the tracking of "Action Frequencies" across different discretized game states.
The bot observes every action the opponent takes and classifies it into a highly specific categorical matrix. In pioneering research by Billings et al., opponent actions were classified into 36 different categories based on the action taken (fold, call/check, raise), the cost of the action (0 bets, 1 bet, >1 bet), and the betting round in which the action occurred (pre-flop, flop, turn, or river).
For every state, the bot calculates a Probability Triple (f, c, r), representing the historical likelihood that the opponent will Fold (f), Call/Check (c), or Raise (r) in that exact situation. Because the probabilities must sum to 1.0, these triples represent a distinct behavioral fingerprint of the opponent.

- An opponent with a pre-flop probability triple of (0.20, 0.60, 0.20) is identified as a "Loose-Passive" player (limping frequently, rarely folding or raising).
- An opponent with (0.70, 0.05, 0.25) is identified as a "Tight-Aggressive" player.

Furthermore, advanced feature extraction can significantly enhance the accuracy of these triples. Studies have shown that utilizing decision trees to analyze features such as the number of suited cards on the board, the player's relative table position, and an overarching "aggressiveness ratio" (the ratio of raises/bets to total actions) yields a highly accurate

predictive model of the opponent's next move.

5.2 Dynamic Range Updating via Bayesian Inference

The primary utility of the opponent model is to dynamically narrow the opponent's assumed hand range from a completely random distribution to a highly targeted subset of cards. At the start of a hand, the opponent's range is assumed to be 100\% of all 1,326 possible starting combinations (or the 169 strategically distinct groupings). As the opponent acts, the bot uses Bayes' Theorem to continually refine this distribution.
The Bayesian update relies on the standard conditional probability formula:
The components of this update are defined as follows:

- **P(Hand_i) (Prior Probability):** The agent's belief that the opponent held a specific hand *before* their current action. At the start of the game, this is a uniform distribution.
- **P(Action | Hand_i) (Marginal/Likelihood Probability):** The likelihood that the opponent would take the observed action given that they hold that specific hand. This is derived directly from the Action Frequency (f, c, r) tables correlated with the EHS of the specific hand.
- **P(Hand_i | Action) (Posterior Probability):** The newly updated weight assigned to the opponent holding that specific hand, which becomes the Prior Probability for the next street.

Operational Example: If an opponent executes a massive 3-bet pre-flop, the Bayesian model evaluates the Marginal Probability of a 3-bet. Historically, the (f, c, r) data dictates that the opponent only raises 5\% of the time. The Bayesian formula heavily inflates the posterior weights of premium hands (AA, KK, QQ, AKs) while drastically reducing the weights of weak, speculative hands, mathematically crushing their probability toward zero.
This continuously updating, weighted probability distribution is then fed directly back into the Monte Carlo simulator. Instead of calculating the EHS against a completely random hand, the Monte Carlo rollouts are heavily biased to select opponent hands according to the Bayesian posterior distribution. This seamless integration of Bayesian inference and Monte Carlo sampling creates a highly lethal exploitative engine that adapts to the opponent's specific strategic leaks within a few dozen hands.

6. Lightweight Decision-Making Frameworks

While the combination of EHS evaluations and Bayesian opponent models creates a robust analytical core, the agent requires a centralized decision-making framework to traverse the game tree and translate these metrics into a final action. Given the exclusion of CFR, two primary frameworks dominate the heuristic space: Case-Based Reasoning and variants of Miximax/Monte Carlo Tree Search.

6.1 Case-Based Reasoning (CBR) and k-Nearest Neighbors

Case-Based Reasoning (CBR) offers an exceptionally fast alternative to live tree search. Pioneered in poker by systems like the CASPER bot, CBR relies on an offline, pre-compiled database of millions of hands played by expert human players or successful bots.
Instead of computing the optimal mathematical path in real-time, the bot analyzes the current state of the table (the board texture, pot size, effective stack, and opponent action) and utilizes a k-Nearest Neighbors (k-NN) algorithm to retrieve the most statistically similar historical situations.
The CBR architecture operates on a cyclical framework known as the 6-REs:

1. **Retrieve:** The bot queries the database for scenarios exceeding a strict similarity threshold (e.g., 97\% similarity in board texture and betting sequence).

2. **Reuse:** It extracts the specific actions taken by the winning players in those historical cases.
3. **Revise & Review:** The bot evaluates the proposed historical action against the current EHS and pot odds.
4. **Retain & Refine:** Post-hand, the bot adds its own resultant successes or failures to the database, allowing the case-base to iteratively improve.

When multiple similar cases are found, the bot averages the historical actions to formulate a probability triple, executing the move that historically yielded the highest Expected Value. CBR requires significant memory overhead to store the case base, but decision latency is virtually zero, making it ideal for the millisecond constraints of bot competitions. If the 100MB compressed file size limit of the MIT Pokerbots competition poses an issue for large databases , the case base can be aggressively pruned to only include high-leverage inflection points (e.g., river all-in scenarios) or highly specific board textures.

6.2 Miximax Search and Information Set MCTS

Standard Minimax search algorithms—used to solve perfect information games like Chess—fail catastrophically in poker. Poker is an imperfect information environment; the bot cannot assume the opponent will execute the mathematically optimal counter-move because the opponent operates under hidden information, and human behavior is inherently stochastic.

To overcome this, developers utilize a specialized game tree variant called **Miximax Search** (or Expectiminimax). In a Miximax game tree, the nodes where the bot acts are standard maximization nodes, seeking the highest EV. However, the nodes where the opponent acts are treated as *chance nodes* rather than minimization nodes. The probability distribution of these chance nodes is populated entirely by the Bayesian opponent model (the tracked action frequencies and probability triples). The bot calculates the expected value of a branch by summing the potential outcomes, weighted by the exact probability that the opponent will take a specific counter-action.

For deeper searches beyond a single street, **Information Set Monte Carlo Tree Search (ISMCTS)** is deployed. ISMCTS operates on sets of states that are indistinguishable to the acting player due to hidden hole cards.

The ISMCTS loop operates in four continuous phases until the microsecond time limit expires:

1. **Selection:** The algorithm traverses the known tree. It balances the exploitation of high-win-rate actions with the exploration of untested lines using the Upper Confidence Bound for Trees (UCT) heuristic: UCT = w_i/n_i + c \sqrt{\ln N / n_i}, where w_i is the number of wins, n_i is the node visits, and N is total visits.
2. **Expansion:** A new node representing a potential action is added to the tree.
3. **Simulation:** The Monte Carlo rollout plays the game out to a terminal state (showdown or fold). Crucially, the simulator assigns randomized hidden cards to the opponent strictly based on the Bayesian posterior range.
4. **Backpropagation:** The resulting win, loss, or tie payoff is propagated back up the tree, updating the expected value of the root decision.

ISMCTS is an "anytime" algorithm. Because it continually refines its estimates rather than requiring a full traversal to return a value, it can be abruptly halted the exact millisecond the 30-second time bank dictates a move must be made, instantly returning the most promising action explored thus far.

7. Adapting to Competition Mechanics: Dealing with Variant Rules

The MIT Pokerbots competition rarely features vanilla Texas Hold'em. To prevent teams from simply downloading pre-solved, open-source CFR matrices (such as those generated by

Pluribus or DeepStack) and submitting them, the competition introduces specialized, proprietary rule variants.
For example, the 2024 competition utilized "Auction Hold'em." This variant introduced a sealed-bid, second-price auction immediately after the flop was dealt. Both players secretly bid portions of their chip stack; the highest bidder paid the second-highest bid amount to the pot and received a third hole card, fundamentally altering standard hand equities.
This is precisely where modular, heuristic architectures drastically outperform rigid CFR architectures in a one-week development timeline. A CFR bot requires the entire game tree and ruleset to be explicitly defined before training begins. Introducing a sealed-bid continuous auction into a CFR game tree causes an exponential state-space explosion, making it entirely unsolvable within a week without massive supercomputing resources and extensive abstraction re-engineering.
Conversely, an architecture built on EHS, Monte Carlo rollouts, and expected value can organically adapt to massive rule changes with minimal friction. In the case of Auction Hold'em, the heuristic bot's evaluation logic simply needs to simulate a 3-card hand versus a 2-card hand. The bot evaluates its current 2-card EHS, runs a parallel Monte Carlo simulation to estimate its 3-card EHS, calculates the marginal increase in win probability, and multiplies that probability by the pot size to determine the exact expected monetary value of the third card. That expected value immediately becomes the mathematical ceiling for its sealed bid. This calculation is executed via the exact same Monte Carlo rollout engine already built for post-flop play, requiring only minor logic tweaks rather than complete retraining.

8. Open-Source Ecosystems and Implementation Pipelines

To successfully build and deploy this complex heuristic architecture within a single week, competitors must leverage highly optimized, open-source libraries rather than coding low-level card evaluation logic from scratch.

8.1 Foundational Engines and Evaluators

The official MIT Pokerbots framework is typically maintained at the mitpokerbots/engine repository, providing the necessary networking scaffolding, TCP/IP protocols, and base skeleton bots in Python, Java, and C++. However, to process millions of Monte Carlo rollouts per second, native Python evaluators are excessively slow.
Competitors widely utilize the pbots_calc repository, a C-based equity calculator provided by the MIT staff, which wraps the legacy poker-eval library for ultra-fast range evaluation and exposes it to Python and Java via wrappers. More modern, high-performance implementations utilize OMPEval or Python-specific wrappers like Deuces and eval7. The OMPEval library, written in highly optimized C++, is capable of evaluating upwards of 400 million random hands per second on a standard CPU. Integrating this library is mandatory to achieve the massive throughput required for deep ISMCTS and Monte Carlo rollouts within a strict 30-millisecond decision window.

8.2 Architectural Blueprints from Past Competitions

Reviewing successful past submissions and open-source projects provides a concrete blueprint for constructing a non-CFR architecture:

- **The Bluffasaurus Repository (petekanev/Bluffasaurus):** Demonstrates the successful application of the full EHS formula (combining HS, PPOT, and NPOT). It limits Monte Carlo subsets to exactly ~2,000 iterations to guarantee execution speeds remain under

0.1 seconds per move.

- **The SPAM_pokerbot_2017 Repository (sebLopezCot/SPAM_pokerbot_2017):** Highlights the power of pre-computing win rates and establishing hard-coded decision rules based on Deuces library scoring. By mapping action heuristics directly to specific board textures and relying heavily on suit symmetries (e.g., treating a flop of 3s-4s-5s identically to 3c-4c-5c), the bot saves massive amounts of runtime compute.
- **The G5 Poker Bot Repository (Nemandza82/g5-poker-bot):** Offers a masterclass in Bayesian opponent modeling in C++. It uses 2 million hands of internet play to establish a baseline prior strategy, and subsequently utilizes Bayesian inference to continually update the opponent's assumed hand range and probability triples, feeding this data directly into an Expectiminimax tree search.
- **The Hugh Pearse Repository (hughpearse/mit-poker-bots):** Provides a clean Python implementation that relies entirely on the Bill Chen formula for pre-flop categorization, transitioning directly into iterative Monte Carlo simulations and Pot Odds calculations for post-flop play.

9. Conclusion

Developing an autonomous poker agent for highly constrained, time-sensitive environments—such as the one-week sprint required by academic competitions like MIT Pokerbots—demands a deliberate rejection of mathematically perfect but computationally monolithic algorithms like Counterfactual Regret Minimization and Deep Reinforcement Learning.

A highly competitive, rapid-deployment architecture relies instead on a synthesis of distinct, lightweight analytical layers. The pre-flop phase is optimized using the deterministic mathematics of the Bill Chen formula and Sklansky-Chubukov push/fold rankings, instantly categorizing hands and isolating computational power for later streets. Post-flop evaluation is driven by the Effective Hand Strength (EHS) metric, approximated through thousands of ultra-fast Monte Carlo rollouts powered by optimized C++ libraries like OMPEval or Deuces.

Crucially, rather than seeking an unexploitable GTO equilibrium, the agent actively exploits sub-optimal adversaries by constructing a dynamic Bayesian opponent model. By tracking action frequencies as probability triples, the agent uses Bayes' theorem to constantly refine the opponent's hand range based on their betting behavior. This refined probability distribution is injected into a Miximax or Information Set Monte Carlo Tree Search (ISMCTS), allowing the bot to calculate the highest expected value against that specific opponent.

Ultimately, this hybrid architecture of mathematical heuristics and localized probabilistic modeling yields an agent that is incredibly fast, computationally inexpensive, highly adaptable to sudden variant rule changes, and lethal against the flawed logic of standard competition opponents.